

GUÍA TÉCNICA · 2026

Multitenancy en LeveyQC

De una petición autenticada a una conexión
del pool correcto para cada laboratorio

Arquitectura observada · razonamiento · ejercicios progresivos

Preparada para Nicolás · uso educativo y replicación aislada

No contiene contraseñas ni secretos del entorno

CÓMO USAR ESTA GUÍA

Aprender el mecanismo, no memorizar clases

El objetivo es que puedas explicar, construir y depurar el recorrido completo sin depender del proyecto original.

Esta guía documenta la arquitectura encontrada en LeveyQC y la convierte en una secuencia de aprendizaje. Primero formarás un modelo mental; después estudiarás cada pieza; finalmente la replicarás en un proyecto nuevo con ejercicios incompletos a propósito.

- Lee las páginas 3 a 20 antes de programar. Dibuja el recorrido de una petición con tus propias palabras.
- Crea un proyecto aislado: no copies paquetes completos. Reproduce una responsabilidad por vez.
- En cada laboratorio escribe primero una hipótesis, ejecuta una prueba y registra lo observado.
- No avances si el criterio de aprobación de la etapa anterior no se cumple.

Resultado esperado
Al terminar podrás responder qué identifica al tenant, quién traduce ese identificador a una conexión, cuándo nace un pool, cómo vuelve una conexión al pool y por qué el contexto debe limpiarse siempre.

Ruta de aprendizaje

Bloque	Páginas	Capacidad
Modelo	3-8	Explicar arquitectura y flujo
Mecanismo	9-19	Seguir tenant, provider y pool
Escalabilidad	20-23	Dar de alta tenants sin redespargar
Práctica	24-32	Construir y experimentar
Referencia	33-36	Depurar y consultar fuentes

FUNDAMENTO

Un laboratorio es un tenant

En LeveyQC, el tenant no es una fila más dentro de una tabla compartida: el diseño apunta a una base de datos separada por laboratorio.

El patrón observado es database per tenant. Existe una base central, LeveyCentral, que conoce usuarios, permisos, laboratorios y la configuración necesaria para encontrar la base de cada laboratorio. Los datos operacionales del laboratorio viven en otra base.

Tres identidades que no debes mezclar

Identidad	Ejemplo conceptual	Responsabilidad
Usuario Clerk	sub del JWT	Demuestra quién inició sesión
Usuario interno	UsuariosLevey	Une identidad externa con reglas propias
Tenant	laboratorioid	Decide qué base recibe la consulta

Regla clave

El token autentica al usuario, pero el backend determina el laboratorio consultando su modelo central. El frontend no debería elegir libremente el identificador de tenant.

Esta separación reduce el riesgo de que un usuario manipule un encabezado como X-Tenant-Id. También concentra la relación usuario-laboratorio en la base central, donde puede ser auditada y modificada.

MAPA DE ARQUITECTURA

Las piezas y sus límites



La identidad del tenant viaja en contexto; las credenciales no viajan en el JWT.

La mitad izquierda representa el plano de control: autenticación, usuarios y catálogo de conexiones. La mitad derecha representa el plano de datos: repositorios y tablas del laboratorio seleccionado.

- Persistencia central usa el DataSource principal y repositorios centrales.
- Persistencia laboratorio usa Hibernate multitency, no un DataSource fijo por entidad.
- LaboratorioContext transporta únicamente laboratoriod durante la petición.
- GestorPoolLaboratorio traduce ese id a una configuración central validada.
- RegistroPoolsLaboratorio reutiliza un HikariDataSource por laboratorio.

SEPARACIÓN JPA

Dos EntityManagerFactory, dos mundos

La separación de paquetes y repositorios evita que una entidad central termine consultándose contra la base de un tenant, o al revés.

Aspecto	Unidad central	Unidad laboratorio
Configuración	PersistenciaCentralConfiguracion	PersistenciaLaboratorioConfiguracion
Persistence unit	central	laboratorio
Transaction manager	transactionManager	laboratorioTransactionManager
Selección DB	DataSource fijo	MultiTenantConnectionProvider
DDL	propiedad principal	hibernate.hbm2ddl.auto = none

Spring Data necesita saber qué repositorio pertenece a qué EntityManagerFactory. Por eso cada @EnableJpaRepositories indica paquetes concretos y referencias explícitas al entityManagerFactory y al transactionManager correspondientes.

Fallo típico

Si un repositorio tenant queda escaneado por la configuración central, puede funcionar en compilación y fallar semánticamente: consultará el esquema incorrecto. La estructura de paquetes también es una barrera de seguridad.

En el proyecto actual, la unidad central es @Primary. Esto resuelve inyecciones ambiguas cuando un componente no declara explícitamente cuál EntityManagerFactory o TransactionManager necesita.

PERSISTENCIA CENTRAL

El catálogo que gobierna a los tenants

PersistenciaCentralConfiguracion construye el EntityManagerFactory central con el DataSource principal y registra siete áreas de repositorios: administradores, asignaciones de permisos, configuración de bases, laboratorios, acciones, tipos de usuario y usuarios Levey.

Su responsabilidad no es devolver datos clínicos. Su responsabilidad es responder preguntas de control:

- ¿El JWT corresponde a un usuario interno conocido?
- ¿Ese usuario es administrador o usuario de laboratorio?
- ¿A qué laboratorio pertenece?
- ¿Existe una configuración de base para ese laboratorio?
- ¿La configuración está activa y habilitada para conexión?

Diseño importante

La configuración central debe estar disponible antes de intentar resolver cualquier conexión tenant. Por eso la persistencia de laboratorio depende del entityManagerFactory central.

Archivo principal:

`src/main/java/cl/leveyqc/leveyqc/Configuracion/PersistenciaCentralConfiguracion.java.`

Ejercicio de lectura: enumera en una hoja qué tablas centrales intervienen antes de una consulta a InformacionLaboratorio. No programes todavía; intenta separar datos de identidad, autorización y conexión.

PERSISTENCIA TENANT

Hibernate recibe dos colaboradores

PersistenciaLaboratorioConfiguracion construye la segunda unidad y entrega a Hibernate dos objetos mediante MultiTenancySettings:

Colaborador	Pregunta que responde
LaboratorioTenantIdentifierResolver	¿Cuál es el tenant de la operación actual?
LaboratorioMultiTenantConnectionProvider	¿Qué conexión JDBC corresponde a ese tenant?

```
properties.put(MULTI_TENANT_IDENTIFIER_RESOLVER, resolver)
properties.put(MULTI_TENANT_CONNECTION_PROVIDER, provider)
properties.put("hibernate.hbm2ddl.auto", "none")
```

La configuración escanea la entidad InformacionLaboratorio y asocia su repositorio al laboratorioEntityManagerFactory. Cuando Spring Data ejecuta findAll(), Hibernate inicia la resolución del tenant antes de obtener la conexión.

Por qué DDL = none

Con múltiples bases no conviene que el arranque intente crear o alterar automáticamente estructuras sin una estrategia explícita. Las migraciones por tenant deben estar controladas y versionadas.

Archivo principal: src/main/java/cl/leveyqc/leveyqc/Configuracion/PersistenciaLaboratorioConfiguracion.java.

RECORRIDO COMPLETO

De fetch() a una conexión física

**Invariante de aislamiento**

La consulta tenant solo ocurre después de identificar al usuario y fijar su laboratorio.

Al terminar la petición, el ThreadLocal se elimina aunque exista una excepción.

El orden importa más que cualquiera de las clases por separado. Un repository no recibe laboratorioId como parámetro: Hibernate lo pregunta al resolver en el momento de trabajar con la sesión.

- El frontend envía Authorization: Bearer <token>.
- Spring Security valida firma, emisor y vigencia del JWT.
- FiltroUsuariosSistema resuelve el actor contra la persistencia central.
- Si es usuario de laboratorio, fija LaboratorioContext con su id.
- El servicio llama al repository tenant.
- Hibernate consulta resolver y provider; el pool entrega una conexión.
- connection.close() devuelve la conexión a HikariCP; finally elimina el ThreadLocal.

ETAPA 1

JWT, actor y autoridad

Autenticación responde quién eres; autorización y multitenancy responden qué puedes hacer y dónde se ejecuta.

FiltroUsuariosSistema extiende OncePerRequestFilter. Obtiene Authentication, lee el JWT y usa ResolutorActorService para localizar al actor interno. Después reemplaza o amplía autoridades de Spring Security.

Actor	Autoridad	Contexto tenant
Administrador	ROLE_ADMIN	No fija laboratorio por defecto
Usuario Levey	ROLE_USUARIO_LEVEY	Fija id desde UsuariosLevey
No reconocido	Sin acceso útil	No debe consultar tenant

Frontera de confianza

El frontend puede mostrar organization.name, pero la organización visual de Clerk no debe sustituir la asociación interna validada por el backend, salvo que diseñes y audites explícitamente esa equivalencia.

La consulta central ocurre antes de poblar el contexto. Así, el tenant nace de datos controlados por el servidor, no de un valor arbitrario enviado por el cliente.

ETAPA 2

LaboratorioContext y el alcance de la petición

LaboratorioContext encapsula un ThreadLocal<Long>. Su contrato es pequeño: setLaboratoriold valida un id positivo, getLaboratoriold lo recupera y clear ejecuta remove().

```
try {
    LaboratorioContext.setLaboratorioId(laboratorioId);
    filterChain.doFilter(request, response);
} finally {
    LaboratorioContext.clear();
}
```

Los servidores reutilizan hilos. Si el id queda asociado después de terminar una petición, el mismo hilo podría atender luego a otro usuario conservando el tenant anterior. Esa es una fuga de aislamiento, no solo una pérdida de memoria.

Invariante

Toda ruta que establezca contexto debe limpiarlo en finally. El finally también se ejecuta si el controlador, servicio, repositorio o serialización lanza una excepción.

Java documenta que un valor ThreadLocal permanece asociado al hilo hasta que se elimina o el hilo termina. En un pool de hilos, el hilo puede vivir durante toda la aplicación; por eso remove() es obligatorio.

Archivo: .../BaseDatosLaboratorio/contexto/LaboratorioContext.java.

ETAPA 3

CurrentTenantIdentifierResolver

LaboratorioTenantIdentifierResolver adapta el contexto propio al contrato de Hibernate.

resolveCurrentTenantIdentifier() lee LaboratorioContext y devuelve el Long. Si no existe, lanza HibernateException.

Lanzar temprano es preferible a usar un tenant por defecto. Un fallback silencioso podría dirigir una operación hacia la base equivocada y ocultar el error de seguridad.

```
resolveCurrentTenantIdentifier():  
id = LaboratorioContext.getLaboratorioId()  
si id es null -> error explícito  
retornar id
```

validateExistingCurrentSessions = true

Hibernate puede comprobar que una sesión ya asociada a un tenant no sea reutilizada con otro identificador. Refuerza la coherencia entre contexto y sesión.

Pregunta de diseño: ¿qué debería ocurrir si un administrador llama a un repositorio tenant sin haber seleccionado un laboratorio? En la arquitectura actual, debe fallar por ausencia de contexto. Para soportar administración cruzada necesitarías un flujo explícito y auditable de selección, no un valor implícito.

ETAPA 4

MultiTenantConnectionProvider

LaboratorioMultiTenantConnectionProvider implementa el punto de extensión JDBC de Hibernate. Recibe un identificador Long, solicita el HikariDataSource correcto al gestor y obtiene una conexión.

Método	Uso en el proyecto
getConnection(id)	Valida id, obtiene pool y llama pool.getConnection()
releaseConnection(id, connection)	Ejecuta close(); Hikari recupera la conexión
getAnyConnection()	Usa DataSource central para metadatos de arranque
releaseAnyConnection(connection)	Cierra/devuelve la conexión de bootstrap
supportsAggressiveRelease()	false; evita liberación agresiva

close() no significa destruir

En una conexión envuelta por HikariCP, close() normalmente la devuelve al pool. El pool decide cuándo cerrar la conexión física según salud y ciclo de vida.

El provider no conoce URLs, contraseñas ni reglas de negocio. Su tarea es traducir tenantId a una Connection usando servicios especializados.

DETALLE DE ARRANQUE

Por qué existe `getAnyConnection()`

Hibernate puede necesitar una conexión antes de que exista una petición y, por lo tanto, antes de que `LaboratorioContext` tenga un id. Ejemplos: inspeccionar metadatos JDBC, dialecto o capacidades del motor.

El provider actual entrega una conexión del `DataSource` central en `getAnyConnection()`. Esto permite el bootstrap sin inventar un tenant predeterminado.

Precaución

La conexión de bootstrap no debe usarse para ejecutar repositorios tenant. Es una conexión auxiliar para inicialización. Mantén `hibernate.hbm2ddl.auto` en `none` y verifica que dialecto/motor sean compatibles entre central y tenants.

Experimento mental

- Si `getAnyConnection()` llamara a `getConnection(null)`, el provider fallaría porque no hay contexto.
- Si escogiera siempre el laboratorio 1, el arranque dependería de que ese tenant estuviera disponible.
- Si central y tenants usaran motores incompatibles, los metadatos podrían inducir una configuración errónea.

La documentación oficial de Hibernate describe `MultiTenantConnectionProvider` como el contrato para adquirir conexiones según el identificador de tenant en estrategias de base o esquema por tenant.

ETAPA 5

GestorPoolLaboratorio valida antes de conectar

GestorPoolLaboratorio es el orquestador. No crea mapas ni configura Hikari directamente; coordina RegistroPoolsLaboratorio, BaseDatosLaboratorioService y FabricaPoolLaboratorio.

Secuencia al pedir un pool

- 1. Rechaza laboratoriod nulo, cero o negativo.
- 2. Pregunta al registro si ya existe un pool para ese id.
- 3. En el primer acceso consulta BaseDatosLaboratorio en LeveyCentral.
- 4. Verifica que la configuración corresponda al mismo laboratorio.
- 5. Exige activo = 1 y estadoConexion = 1.
- 6. Delega a la fábrica la creación del HikariDataSource.
- 7. Conserva el pool para siguientes solicitudes.

Defensa en profundidad

Aunque el id proviene del usuario central, el gestor vuelve a validar la configuración y su estado. No basta con que una fila exista: debe estar habilitada para conexión.

Este diseño permite invalidar un pool cuando cambia una configuración. Sin invalidación, el mapa conservaría conexiones con credenciales o destino antiguos.

ETAPA 6

RegistroPoolsLaboratorio: caché concurrente

RegistroPoolsLaboratorio mantiene ConcurrentHashMap<Long, HikariDataSource>. obtenerOCrearPool usa computeIfAbsent: para una clave ausente, calcula y registra el pool de forma atómica.

```
pool = pools.computeIfAbsent(laboratorioId, id -> creador.apply(id))
```

Por qué esto importa

- Dos peticiones simultáneas del mismo laboratorio no deberían dejar dos pools permanentes.
- Laboratorios distintos mantienen pools separados y pueden inicializarse bajo demanda.
- No se crean conexiones para laboratorios que todavía no han recibido tráfico.
- invalidarPool remueve y cierra un pool específico.
- @PreDestroy cierra todos los pools al apagar la aplicación.

Escala y límites

Un pool por tenant multiplica conexiones potenciales. Con 100 tenants y máximo 5, el techo teórico es 500 conexiones. Debes dimensionar base, aplicación y políticas de inactividad; no asumir que el mapa es gratuito.

computeIfAbsent evita una carrera básica, pero la creación debe ser rápida, determinista y no reentrante para esa clave. Los fallos deben propagarse sin almacenar un pool inválido.

ETAPA 7

FabricaPoolLaboratorio construye HikariCP

La fábrica recibe BaseDatosLaboratorio, valida host, puerto, nombre de base, usuario y clave lógica del secreto. Luego busca la contraseña en Environment y construye la URL JDBC de MySQL.

Propiedad actual	Valor	Efecto
maximumPoolSize	5	Hasta cinco conexiones totales por tenant
minimumIdle	1	Mantiene al menos una conexión inactiva
connectionTimeout	10 s	Espera máxima por una conexión
validationTimeout	5 s	Límite para validar salud
idleTimeout	300 s	Retiro de inactivas sobre minimumIdle
maxLifetime	1.800 s	Renovación preventiva de conexiones

El override LEVEY_TENANT_HOST_OVERRIDE permite reemplazar el host registrado, útil cuando el mismo catálogo se ejecuta desde Docker o directamente en macOS y el nombre host.docker.internal cambia de significado.

No registres contraseñas

Loguea laboratoriod, nombre de pool y destino sanitizado. Nunca imprimas password, JWT, variables secretas ni URLs que las contengan.

CONEXIÓN Y TRANSACCIÓN

Qué se presta y qué se devuelve

Un pool no asigna una única conexión permanente a cada laboratorio. Asigna un pool independiente por laboratorio; dentro de ese pool, cada operación toma temporalmente una conexión disponible.

Momento	Objeto	Estado
Primer acceso tenant	HikariDataSource	Se crea y queda registrado
Inicio operación JPA	Connection proxy	Prestada desde el pool
Durante transacción	Connection	Asociada al EntityManager/transacción
Fin/rollback	Connection.close()	Devuelta al pool
Invalidación/apagado	HikariDataSource.close()	Cierra conexiones físicas

Distinción

releaseConnection cierra el préstamo; invalidarPool cierra el pool. Confundir ambos niveles puede provocar fugas o destruir el rendimiento de reutilización.

El laboratorioTransactionManager debe gobernar operaciones de repositorios tenant. Si una operación combina escrituras central y tenant, no existe atomicidad distribuida automática entre ambas bases. Debes diseñar consistencia, compensaciones o mensajería según el caso.

EJEMPLO REAL

GET /informacionlaboratorio

El frontend obtiene un token con `getToken()` y envía `Authorization: Bearer`. Si Spring Security lo acepta, el filtro fija el contexto; el controlador llama a `InformacionLaboratorioService`; el servicio usa `InformacionLaboratorioRepository.findAll()`.

```
fetch(API + "/informacionlaboratorio", {  
  headers: { Authorization: "Bearer " + token }  
})
```

La llamada parece un CRUD normal porque el multitenancy está debajo del repositorio. Esa transparencia es útil, pero obliga a proteger todas las entradas: un repositorio tenant ejecutado fuera de una petición correctamente contextualizada debe fallar.

Detalle en el frontend mostrado

No puedes leer primero `res.text()` y después `res.json()` sobre la misma respuesta; el body se consume una vez. La segunda versión que usa directamente `res.json()` cuando `res.ok` es correcta para el caso de éxito.

También conviene comprobar que `NEXT_PUBLIC_API_URL` esté definido y que el backend devuelva JSON coherente en éxitos y errores. Estas mejoras no cambian el mecanismo multi-tenant, pero facilitan el diagnóstico.

ESTADO COMPROBADO

Qué está listo y qué depende del entorno

Durante la revisión del proyecto, la compilación Maven sin descargar dependencias finalizó correctamente. Las pruebas de contexto Spring inicializaron las dos unidades de persistencia: siete repositorios centrales y un repositorio tenant.

Comprobación	Resultado	Lectura
Compilación	Correcta	Clases y configuración son compatibles
Arranque contexto	Correcto	Dos EntityManagerFactory se crean
Endpoint + JWT	Reportado funcional	Flujo real llega al tenant
Prueba directa macOS	Host no resuelto	Diferencia red Docker/host

Diagnóstico macOS

host.docker.internal está pensado para alcanzar el host desde un contenedor. Si la aplicación Java corre directamente en macOS, suele corresponder localhost o la dirección real. LEVEY_TENANT_HOST_OVERRIDE permite probar sin alterar el catálogo.

No confundas un fallo de DNS o red con un fallo del resolver de Hibernate. Sigue la cadena: contexto -> configuración encontrada -> pool creado -> host resuelto -> autenticación MySQL -> consulta SQL.

ALTA DE TENANT HOY

Qué debe existir para un laboratorio nuevo

Con la arquitectura actual, un nuevo laboratorio necesita datos de control en LeveyCentral, una base operativa creada y un secreto accesible por la aplicación. El pool no necesita declararse manualmente: se crea bajo demanda.

Elemento	Ubicación	¿Requiere redespiegue?
Laboratorio	LeveyCentral	No
Relación usuario-lab	LeveyCentral	No
Host/puerto/base/usuario	BaseDatosLaboratorio	No
Esquema y datos iniciales	Nueva BD tenant	Depende del proceso
Password actual	Variable .env referenciada	Sí, normalmente

La columna secretoConexionKey guarda el nombre lógico de la variable, no la contraseña. Por ejemplo, una fila puede señalar DB_PASSWORD_LAB_XX y la aplicación busca ese nombre en Environment.

Respuesta a tu preocupación

Sí: hoy el alta sigue acoplada al entorno porque cada contraseña nueva exige una variable disponible para el proceso. El catálogo ya es dinámico; el almacenamiento de secretos es la parte que falta desacoplar.

EVOLUCIÓN ESCALABLE

De .env por tenant a un catálogo seguro

Puedes dejar la configuración operativa en BaseDatosLaboratorio, pero no conviene guardar contraseñas en texto plano. Hay tres niveles de evolución:

Opción	Ventaja	Riesgo / coste
A. .env por tenant	Simple y actual	Reinicio, despliegue y crecimiento manual
B. secreto cifrado en BD	Alta dinámica	Debes custodiar clave maestra y rotación
C. gestor de secretos	Auditoría, rotación, acceso dinámico	Infraestructura y dependencia externa

Diseño recomendado: BaseDatosLaboratorio almacena un secretRef estable. Un ServicioSecretos resuelve esa referencia desde Vault, AWS Secrets Manager, Google Secret Manager u otro almacén. La fábrica recibe la contraseña en memoria, configura Hikari y descarta la referencia local.

Separación de responsabilidades

La base central conoce dónde está el secreto; el gestor de secretos conoce su valor; los logs no conocen ninguno de los dos valores sensibles.

Si eliges cifrado en BD, usa cifrado autenticado, clave maestra fuera de la BD, versionado de claves, rotación y control de acceso. Codificar en Base64 no es cifrar.

FLUJO PROPUESTO

Alta sin editar .env

Esta página describe una evolución, no el estado actual. Implementa cada paso solo después de diseñar permisos, rollback y auditoría.

- 1. Un administrador autorizado registra el laboratorio.
- 2. Un proceso de aprovisionamiento crea la base y aplica migraciones versionadas.
- 3. Se crea un usuario de base con privilegios mínimos para ese esquema.
- 4. La contraseña se guarda en el gestor de secretos y devuelve secretRef.
- 5. BaseDatosLaboratorio guarda host, puerto, databaseName, username y secretRef.
- 6. Una prueba de conectividad abre y cierra una conexión sin exponer el secreto.
- 7. Se marca activo/estadoConexion solo después de la prueba.
- 8. El primer tráfico crea el pool bajo demanda.

Actualización

Cuando cambies host, usuario o secreto, invalida el pool del laboratorio. La siguiente petición lo recreará con la configuración nueva.

Piensa el alta como una transacción de negocio con estados: PENDIENTE, APROVISIONANDO, VALIDANDO, ACTIVO y ERROR. Evita que un tenant parcialmente configurado aparezca como disponible.

CHECKLIST DE PRODUCCIÓN

Aislamiento, capacidad y operación

Área	Pregunta de control
Identidad	¿laboratoriold proviene del backend y no del cliente?
Contexto	¿clear() corre en finally en todos los caminos?
Repositorios	¿paquetes central/tenant están separados?
Conexiones	¿cada pool tiene nombre, límites y timeouts?
Secretos	¿no aparecen en BD plana, código o logs?
Migraciones	¿todos los tenants conocen su versión de esquema?
Observabilidad	¿métricas etiquetan tenant sin datos sensibles?
Capacidad	¿suma de maximumPoolSize cabe en MySQL?
Cambios	¿actualizar config invalida el pool?

Riesgo arquitectónico

database-per-tenant mejora aislamiento, pero multiplica migraciones, monitoreo, backups, pools y recuperación. La automatización operativa es parte del diseño, no un detalle posterior.

Antes de producción, añade pruebas negativas: usuario sin laboratorio, laboratorio inactivo, secretRef inexistente, credencial inválida, host caído, pool agotado y excepción que compruebe la limpieza del contexto.

LABORATORIO 0

Preparar un proyecto laboratorio

Construir un entorno descartable donde cada experimento sea observable y no afecte LeveyQC.

Preparación

Crea un proyecto Spring Boot nuevo con Java 17, Web, Data JPA, Security, OAuth2 Resource Server, MySQL y HikariCP. Usa Git desde el primer momento. No copies el proyecto original.

Trabajo de Nicolás

- 1. Levanta una base central y dos bases tenant con nombres evidentes: tenant_rojo y tenant_azul.
- 2. Crea en ambas tenants una tabla mensaje con la misma estructura, pero contenido distinto.
- 3. Configura únicamente la persistencia central y confirma que el contexto Spring arranca.
- 4. Documenta puertos, usuarios y red en un README; nunca incluyas contraseñas reales.

Qué deberías observar

- Una consulta manual a cada base devuelve mensajes diferentes.
- El proyecto arranca aunque todavía no exista multitенancy.
- Puedes destruir y reconstruir el entorno sin pasos secretos.

Criterio de aprobación

Tienes tres bases aisladas, un arranque verde y un diagrama propio de una página.

Preguntas para razonar

- ¿Qué información pertenece a central?
- ¿Qué dato demuestra visualmente que no mezclaste tenants?

Pista graduada

Empieza con SQL mínimo. El objetivo no es el dominio clínico, sino hacer visible el enrutamiento.

LABORATORIO 1

Modelar el catálogo central

Aprender a convertir laboratoriod en una configuración de conexión sin abrir todavía conexiones dinámicas.

Preparación

En la base central crea tablas mínimas laboratorio, usuario_interno y base_datos_laboratorio. Usa campos de estado y una referencia de secreto ficticia.

Trabajo de Nicolás

- 1. Inserta dos laboratorios activos y uno inactivo.
- 2. Asocia dos usuarios ficticios a laboratorios diferentes.
- 3. Escribe consultas que, dado un subject externo, devuelvan exactamente un laboratorio activo.
- 4. Escribe pruebas para id inexistente, inactivo y configuración perteneciente a otro laboratorio.

Qué deberías observar

- La consulta central nunca toca las tablas mensaje.
- Un laboratorio inactivo no produce una configuración utilizable.
- La referencia de secreto es un nombre, no una contraseña.

Criterio de aprobación

Las pruebas de catálogo cubren éxito y tres fallos sin construir ningún HikariDataSource.

Preguntas para razonar

- ¿Dónde validarías que configuración.laboratorio_id coincide con el solicitado?
- ¿Qué restricción UNIQUE evitaría ambigüedad?

Pista graduada

Separa el repositorio que lee filas del servicio que decide si una configuración puede usarse.

LABORATORIO 2

Experimentar con ThreadLocal

Comprobar por ti mismo por qué el contexto debe limpiarse al reutilizar hilos.

Preparación

Crea una clase TenantContext mínima con set, get y clear. Todavía no uses Spring ni Hibernate. Ejecuta tareas sobre un ExecutorService de un solo hilo.

Trabajo de Nicolás

- 1. Tarea A fija tenant 10 y termina sin clear; tarea B solo lee.
- 2. Registra qué ve B y explica por qué.
- 3. Repite con try/finally y clear.
- 4. Agrega una excepción intencional antes de terminar A y verifica que finally limpia.

Qué deberías observar

- Sin clear, B puede heredar 10 porque usa el mismo hilo.
- Con clear, B obtiene null.
- La excepción no impide la limpieza cuando existe finally.

Criterio de aprobación

Puedes mostrar una prueba roja sin clear y la misma prueba verde con clear.

Preguntas para razonar

- ¿ThreadLocal identifica usuarios o solo asocia un valor al hilo?
- ¿Qué cambia si el executor tiene diez hilos?

Pista graduada

Fuerza un pool de un solo hilo para que la reutilización sea determinista y didáctica.

LABORATORIO 3

Construir el registro de pools

Entender lazy loading, concurrencia, invalidación y cierre sin involucrar Hibernate.

Preparación

Crea un registro basado en ConcurrentHashMap y una fábrica contadora que no abra todavía conexiones reales.

Trabajo de Nicolás

- 1. Solicita cinco veces el pool del tenant 1 y cuenta cuántas veces se invoca la fábrica.
- 2. Solicita tenants 1 y 2 desde tareas concurrentes.
- 3. Invalida tenant 1 y confirma que el recurso anterior se cierra.
- 4. Solicita tenant 1 otra vez y confirma que nace una instancia nueva.

Qué deberías observar

- La fábrica corre una vez por tenant mientras la entrada siga vigente.
- La invalidación afecta solo a su clave.
- El cierre global recorre y cierra todos los recursos.

Criterio de aprobación

Tus pruebas demuestran creación única, separación por clave e invalidación recreable.

Preguntas para razonar

- ¿Qué sucede si la fábrica lanza una excepción?
- ¿Debe quedar una entrada inválida en el mapa?

Pista graduada

Usa un recurso falso con boolean closed. Cuando la semántica esté verde, reemplázalo por HikariDataSource.

LABORATORIO 4

Crear pools Hikari reales

Traducir una configuración validada a un pool pequeño y observar su ciclo de vida.

Preparación

Conecta la fábrica a tenant_rojo y tenant_azul. Limita maximumPoolSize a 2 para que los experimentos sean visibles.

Trabajo de Nicolás

- 1. Abre una conexión del pool rojo y consulta el mensaje distintivo.
- 2. Cierra el préstamo y vuelve a abrir; observa métricas del pool.
- 3. Ocupa dos conexiones y solicita una tercera con connectionTimeout corto.
- 4. Cambia una credencial ficticia, invalida el pool y comprueba recreación o fallo controlado.

Qué deberías observar

- close() reduce conexiones activas sin destruir inmediatamente el pool.
- La tercera solicitud espera y termina por timeout cuando el pool está agotado.
- Un pool inválido no debe quedar disponible silenciosamente.

Criterio de aprobación

Puedes distinguir préstamo, conexión física, pool e invalidación mediante logs sanitizados.

Preguntas para razonar

- ¿Cuántas conexiones máximas existirían con 50 tenants activos?
- ¿Qué timeout ayuda a fallar rápido bajo saturación?

Pista graduada

Consulta HikariPoolMXBean para observar active, idle, total y threadsAwaitingConnection.

LABORATORIO 5

Implementar resolver y provider

Conectar el identificador contextual con el registro de pools mediante los contratos de Hibernate.

Preparación

Crea tus propias implementaciones de `CurrentTenantIdentifierResolver` y `MultiTenantConnectionProvider`. Usa TODOS; no copies las clases de LeveyQC.

Trabajo de Nicolás

- 1. Haz que el resolver falle si el contexto es null.
- 2. Haz que `getConnection(id)` pida el pool al gestor y una conexión al pool.
- 3. Haz que `releaseConnection` devuelva el préstamo.
- 4. Diseña `getAnyConnection` para bootstrap y documenta por qué no es un tenant por defecto.

Qué deberías observar

- Sin contexto, la excepción aparece antes de una consulta SQL tenant.
- Con contexto rojo, la conexión proviene solo del pool rojo.
- Cambiar el contexto antes de crear una nueva sesión cambia el destino.

Criterio de aprobación

Pruebas unitarias verifican id nulo, selección correcta y devolución de la conexión.

Preguntas para razonar

- ¿Por qué el provider no debería consultar directamente el JWT?
- ¿Qué responsabilidad quedaría mezclada si construye URLs?

Pista graduada

Prueba primero con dobles de `GestorPool`. Integra MySQL solo cuando el contrato esté claro.

LABORATORIO 6

Separar dos unidades JPA

Reproducir el límite central/tenant y evitar el escaneo cruzado de repositorios.

Preparación

Organiza paquetes central y tenant. Crea una entidad de catálogo central y una entidad Mensaje tenant. Define dos EntityManagerFactory y dos TransactionManager.

Trabajo de Nicolás

- 1. Asocia explícitamente cada @EnableJpaRepositories con su fábrica y transacción.
- 2. Inyecta las propiedades del resolver y provider solo en la unidad tenant.
- 3. Ejecuta repositorio central sin contexto y confirma que funciona.
- 4. Ejecuta repositorio tenant sin contexto y confirma que falla; luego con rojo y azul.

Qué deberías observar

- Central no depende de TenantContext.
- Tenant devuelve contenidos distintos con el mismo método findAll().
- Mover intencionalmente un repositorio al paquete incorrecto reproduce un fallo explicable.

Criterio de aprobación

Tres pruebas verdes: central sin tenant, rojo aislado y azul aislado; una prueba negativa sin contexto.

Preguntas para razonar

- ¿Qué bean debería ser @Primary?
- ¿Qué transactionManager necesita un servicio tenant?

Pista graduada

Dibuja los paquetes antes de crear clases. El escaneo es parte del diseño, no una decoración.

LABORATORIO 7

Contextualizar una petición HTTP

Completar el recorrido desde identidad autenticada hasta repositorio y demostrar la limpieza al finalizar.

Preparación

Usa tokens de prueba o un JwtDecoder controlado. La identidad externa debe resolverse contra tu tabla central antes de fijar el tenant.

Trabajo de Nicolás

- 1. Implementa un filtro por petición que limpie al inicio y en finally.
- 2. Prueba usuario rojo, usuario azul, usuario inexistente y laboratorio inactivo.
- 3. Crea un endpoint que lance una excepción después de acceder al tenant.
- 4. En la siguiente petición del mismo hilo, confirma que no quedó el id anterior.

Qué deberías observar

- El cliente nunca decide el tenant mediante un encabezado libre.
- Cada token aceptado devuelve exclusivamente el mensaje de su laboratorio.
- Un fallo HTTP no contamina la petición siguiente.

Criterio de aprobación

Las pruebas de integración cubren dos éxitos y al menos tres fallos de aislamiento.

Preguntas para razonar

- ¿Qué cambia entre 401 y 403?
- ¿Dónde ubicarías un correlationId para seguir la petición sin registrar el token?

Pista graduada

Para probar fuga, configura el servidor de pruebas con pocos hilos o invoca directamente el filtro en el mismo hilo.

MATRIZ DE EXPERIMENTOS

Romper el sistema de forma controlada

Experimento	Predicción	Evidencia a capturar
Contexto null	Hibernate rechaza la operación	Tipo y punto de excepción
Tenant inactivo	Gestor no crea pool	Contador de fábrica = 0
secretRef ausente	Fábrica falla antes de JDBC	Mensaje sanitizado
Password inválida	Hikari no autentica	SQLState sin contraseña
Host inválido	Fallo DNS/conexión	Destino y tiempo
Pool agotado	Timeout controlado	threadsAwaitingConnection
Config actualizada	Pool viejo persiste	Necesidad de invalidar
Excepción HTTP	finally limpia	Contexto null después
50 tenants	crecen pools/conexiones	métricas totales

Método científico

- Antes de ejecutar, escribe una predicción falsable.
- Cambia una sola variable por experimento.
- Captura métricas, logs sanitizados y resultado SQL.
- Explica el resultado usando el recorrido de siete etapas.
- Restaura el entorno y repite para comprobar que no fue casualidad.

Meta pedagógica

Si solo haces funcionar el caso feliz, conoces la receta. Cuando puedes predecir dónde y cómo falla cada capa, entiendes la arquitectura.

PLAYBOOK DE DEPURACIÓN

Localizar el fallo por capas

Síntoma	Primera comprobación	Capa probable
401	firma, issuer, expiración	Autenticación JWT
403	autoridades y actor	Autorización
No tenant identifier	LaboratorioContext	Filtro/resolver
Config no encontrada	fila central y estados	Catálogo/gestor
UnknownHost	host desde runtime actual	DNS/red
Access denied MySQL	usuario/secretRef	Credenciales
Connection timeout	active/idle/awaiting	Capacidad pool
Table not found	migración/esquema	Provisionamiento
Datos de otro lab	origen id + limpieza	Aislamiento crítico

Orden de investigación: token aceptado -> actor encontrado -> laboratoriod fijado -> resolver devuelve id -> gestor encuentra configuración -> fábrica resuelve secreto -> red conecta -> SQL usa esquema esperado -> contexto se limpia.

Ante mezcla de datos

Detén la prueba y trata el incidente como seguridad. Captura correlationId, usuario interno, laboratorio esperado, pool seleccionado y transacción, sin registrar secretos ni datos clínicos.

PROYECTO FINAL

Aprovisionamiento dinámico de tres tenants

Construye una demostración completa en la que un administrador registra un tercer tenant sin reiniciar la aplicación. Hazlo por etapas y conserva un historial de decisiones.

Requisitos

- Dos persistencias JPA separadas y paquetes no superpuestos.
- Tenant derivado de identidad interna, nunca de un header libre.
- Pool Hikari bajo demanda, invalidable y observable.
- Referencia de secreto desacoplada del .env por tenant.
- Migración versionada aplicada antes de activar el tenant.
- Pruebas de aislamiento paralelas para tres usuarios.
- Prueba de rotación de secreto que invalida y recrea el pool.
- Documento de recuperación ante base tenant no disponible.

Defensa oral

Explicale a otra persona el sistema sin abrir el código. Si puedes dibujar el flujo, justificar cada límite y anticipar cinco fallos, la comprensión ya es transferible.

Entrega sugerida: diagrama, README reproducible, colección HTTP, pruebas automatizadas, tabla de riesgos y una página de retrospectiva con decisiones que cambiarías.

MAPA DE REFERENCIA

Clases actuales y responsabilidad única

Clase	Responsabilidad
PersistenciaCentralConfiguracion	EntityManagerFactory y repos centrales
PersistenciaLaboratorioConfiguracion	Unidad JPA tenant y propiedades Hibernate
FiltroUsuariosSistema	Actor, autoridad, contexto y limpieza
LaboratorioContext	ThreadLocal del laboratorio actual
LaboratorioTenantIdentifierResolver	Expone tenantId a Hibernate
LaboratorioMultiTenantConnectionProvider	Entrega Connection por tenant
GestorPoolLaboratorio	Valida y coordina lookup/creación
RegistroPoolsLaboratorio	Caché concurrente e invalidación
FabricaPoolLaboratorio	Construye y configura HikariDataSource
BaseDatosLaboratorio	Metadatos de conexión en central

Deuda técnica visible

El paquete InfomracionLaboratorio contiene un error tipográfico. No rompe por sí mismo si todo referencia el mismo paquete, pero dificulta búsquedas, convenciones y mantenimiento; conviene corregirlo mediante refactor seguro.

Glosario: tenant, cliente lógico aislado; DataSource, fábrica de conexiones; pool, conjunto reutilizable; EntityManagerFactory, creador de contextos JPA; bootstrap, inicialización; secretRef, referencia a un secreto externo; lazy, creación al primer uso.

FUENTES Y SIGUIENTES PASOS

Referencias primarias

Código del proyecto revisado

- src/main/java/cl/leveyqc/leveyqc/Configuracion/PersistenciaCentralConfiguracion.java
- src/main/java/cl/leveyqc/leveyqc/Configuracion/PersistenciaLaboratorioConfiguracion.java
- src/main/java/cl/leveyqc/leveyqc/BaseDatosLaboratorio/hibernate/
- src/main/java/cl/leveyqc/leveyqc/BaseDatosLaboratorio/pool/
- src/main/java/cl/leveyqc/leveyqc/BaseDatosLaboratorio/contexto/LaboratorioContext.java
- src/main/java/cl/leveyqc/leveyqc/Configuracion/FiltroUsuariosSistema.java
- src/test/java/.../InformacionLaboratorioRepositoryTest.java

Documentación oficial consultada

- Hibernate ORM User Guide: docs.hibernate.org/orm/current/userguide
- Hibernate Introduction: docs.hibernate.org/stable/orm/introduction
- Spring Boot EntityManagerFactoryBuilder: docs.spring.io/spring-boot/api
- HikariCP README: github.com/brettwooldridge/HikariCP
- Java ThreadLocal: [Oracle Java Core Libraries Guide](https://docs.oracle.com/javase/8/docs/api/java/lang/ThreadLocal.html)
- OWASP Secrets Management Cheat Sheet: cheatsheetseries.owasp.org
- HashiCorp Vault database secrets: developer.hashicorp.com/vault

Cierre

La arquitectura base ya resuelve la selección dinámica y el pool por laboratorio. El salto de escalabilidad pendiente es automatizar aprovisionamiento, migraciones y secretos para que el alta no dependa de editar el entorno y redespargar.

Versión documental: 1.0 · septiembre de 2026. Este material evita exponer valores sensibles y distingue explícitamente arquitectura observada de propuestas de evolución.